

PowerShell Cyber Defense Operations

Part 2 Activity: Core Mechanics, Variables, & The Pipeline

This workbook merges all core mechanics, structural discovery rules, mid-level challenges, and the fully bulletproofed parent-child threat-hunting scenario for cyber defense operations.

1. Understanding the Pipeline (|)

Think of the pipeline like passing a baton in a relay race. The first command crosses its finish line and hands its data results directly to the next runner (the next command) without stopping or saving anything to your hard drive.

2. Variables (\$) and the Current Object (\$_)

Variables (\$): A dollar sign followed by a name is a temporary storage container. You can stuff text, numbers, or entire system logs into a variable to reuse later. **Crucial Rule: Never name your own variable \$PID!** This is an automatic PowerShell variable that stores your current active PowerShell process ID. Use names like \$targetPid or \$malwarePid instead.

The Current Pipeline Object (\$_): This represents "the specific item currently in your hands". When you pass a list of items through a pipeline, PowerShell looks at them one by one. The symbol \$_ allows you to inspect or filter that exact single item.

3. Single Quotes vs. Double Quotes & Subexpressions \$()

Single Quotes (' '): What you see is what you get. It treats text strictly as literal text.

Double Quotes (" "): These are smart quotes. They look inside the text for variables or expressions and expand (extrapolate) them to show their true value.

Subexpressions \$(): If you want to execute a mini-command or grab a specific property of an object inside double quotes, you wrap it in \$().

The following table demonstrates quote expansion and subexpressions behavior:

Command Example	Expected Output / Behavior
\$name = "Student"	Stores the word "Student" inside the variable \$name.
'Hello \$name'	Outputs literally: Hello \$name
"Hello \$name"	Outputs dynamically: Hello Student

Command Example	Expected Output / Behavior
<code>\$today = Get-Date</code>	Stores the exact current time object into <code>\$today</code> .
<code>"Year: \$today.Year"</code>	Incorrect. Outputs the date string followed by literal text: Year: 06/24/2026 17:05:00.Year
<code>"Year: \$(\$today.Year)"</code>	Correct. Executes the code inside <code>\$()</code> and outputs the actual current year: Year: 2026

4. Step-by-Step Activity: Command Discovery & Telemetry

IMPORTANT PRE-REQUISITE NOTE: Before starting, your system's help files might be blank. Run `Update-Help -ErrorAction SilentlyContinue` in your terminal. This downloads the latest official Microsoft manuals. We include `-ErrorAction SilentlyContinue` so that if certain non-essential modules fail, the system suppresses the messy red error text and continues installing smoothly.

Step 1: Find All Commands That Use a Specific Action (Verb)

PowerShell uses a strict Verb-Noun pattern. The verb tells PowerShell what action to perform. To see every command capable of "getting" or retrieving information, search by the verb.

Command	Description
<code>Get-Command -Verb Get</code>	Filters the entire system to show only commands meant for retrieving data.

Step 2: Find All Commands Targeting a Component (Noun) or Module

The noun tells PowerShell what object or resource to target.

What is a Module? A module is a folder containing a collection of related commands packed together (like a tool belt dedicated entirely to Firewalls or specific settings). It helps you narrow down your search strictly to target tools instead of scrolling through thousands of unrelated commands.

Command	Description
<code>Get-Command -Noun Process</code>	Searches your session for any command built specifically to manage system processes.
<code>Get-Command -Module NetAdapter</code>	Finds all commands grouped inside a specific operational module.

Step 3: Global Keyword Search (Verb or Noun)

When you don't know the exact cmdlet name, use wildcards (*) to hunt for any command containing your keyword anywhere in its name.

Command	Description
<code>Get-Command *Process*</code>	Acts as a search engine to find any process-related command, regardless of where the keyword falls.

Step 4: View Basic Help Documentation

Syntax Pattern: `Get-Help <Cmdlet-Name>`

Command	Description
<code>Get-Help Get-Process</code>	Pulls open the standard help file for the target cmdlet.

Step 5: Read Comprehensive Details, Full Manuals, & Examples

To truly understand a command, you can ask for deep operational details, the entire unbroken manual, or copy-pasteable real-world examples.

Command	Description
<code>Get-Help Get-Process -Detailed</code>	Shows parameter descriptions alongside execution examples.
<code>Get-Help Get-Process -Full</code>	Opens the entire, completely unrestricted help manual containing all deep technical rules.
<code>Get-Help Get-Process -Examples</code>	Jumps straight to the practical example section so you can see the command in action.

Step 6: Pipeline Discovery (Search & Learn)

Instead of searching for a command and typing it again, you can pass your discovery result directly over to the help engine using the pipeline.

Command	Description
<code>Get-Command Get-Process Get-Help</code>	Discovers the command object and hands it straight to Get-Help to read its documentation instantly.

Step 7: Inspect Hidden Data Properties (Dealing with Objects)

PowerShell output is not flat text; it is made up of Objects (structured digital blocks of data). For example, Get-Process returns process objects, and each object contains hidden properties (like Id, CPU, or Path). To build an effective data filter, use Get-Member to reveal all those hidden attributes.

Command	Description
Get-Process Get-Member	Intercepts the process data objects and exposes all underlying properties available for filtering.

Step 8: Isolating and Choosing Specific Columns

Once you know the property names from Get-Member, use the pipeline to clean up your terminal by selecting only the specific data properties you care about.

Command	Description
Get-Process Select-Object Id, ProcessName	Drops all unnecessary data clutter, rendering a clean, customized data table.

Step 9: Precision Filtering with Conditions

A threat hunter searches for explicit anomalies. Use Where-Object along with the pipeline object variable (\$_) to check a condition on every single object passing through.

Command	Description
Get-Process Where-Object {\$_.ProcessName -eq "cmd"}	Evaluates each process object; if the current object's (\$) process name property exactly equals "cmd", it lets it pass through.

Step 10: Sort and Rank System Evidence

To identify potential malicious persistence or system stress, sort your results by data properties like CPU utilization.

Command	Description
Get-Process Sort-Object CPU -Descending	Organizes the live objects dynamically, placing the heaviest resource consumers at the very top.

Step-by-Step Activity 11: Challenge Mode — Build the Master Investigative Chain!

Now it is your turn to piece the puzzle together. Write a single pipeline command that performs a true system audit based on the following security instructions:

- **Filter:** Isolate only the processes where the CPU usage property is greater than 10.
- **Sort:** Order those results from highest to lowest CPU usage (Descending).
- **Select Columns:** Extract only the three columns needed for a clean incident response report.

 **HINT:** The three columns you need are the process name, the process identification number, and the CPU property. If you aren't sure of their exact spelling, check back on your Step 7 (Get-Member) results to ensure your parameters match perfectly!

Your Challenge Answer Code Goes Here:



Final Boss Challenge: Operation Obscured Beacon

This final challenge block contains your entire security incident scenario, setup script, and investigative phases. Read the background context carefully, execute the setup, and use your PowerShell pipeline skills to hunt down the threat and clean up the system.

The Security Incident Scenario

Your company's Endpoint Detection and Response (EDR) alert system has just flagged a critical anomaly: an unusual flurry of notepad.exe windows are opening minimized in the background. As the SOC analyst on duty, you cannot rely on a known file name or a basic signature. You must trace these windows back to their origin, audit the parent process, extract the adversary's hidden indicators, and completely neutralize the threat.

Step 1: Initialize the Threat (Setup)

To simulate this live attack, copy and paste the exact command below into your standard terminal and press Enter. This launches the background routine that mimics an active piece of malware attempting automated beaconing behavior:

```
powershell -WindowStyle Hidden -Command  
"$Tag='Rm9yZXN0IGlzIENvb2tpbmCGT2JhaSBjZGVhIQ=='; while($true){ Start-Process  
notepad -WindowStyle Minimized; Start-Sleep -Seconds 8 }"
```

(Note: Once executed, the process will run entirely hidden in the background, and notepad windows will silently start to populate your system minimized.)

Step 2: The Investigation Assignment

 **New Tool Alert: Get-CimInstance:** Up until now, we have relied on Get-Process. However, standard process views do not always expose process ancestry out of the box. To level up our

forensic capabilities, we will explicitly use `Get-CimInstance Win32_Process`. This cmdlet queries the deep Windows Management Instrumentation (WMI) repository, which tracks low-level operating system telemetry—including the exact parent IDs responsible for creating applications.

Phase A: Trace the Outbreak

You know that `notepad.exe` instances are spawning rapidly. Start your investigation by querying the deep system repository using `Get-CimInstance Win32_Process` to find out exactly how they were created.

Write a single pipeline command that:

- Queries all processes using `Get-CimInstance Win32_Process`
- Filters the list to isolate only the objects whose name property exactly matches "`notepad.exe`".
- Selects three specific columns to print: the name of the process, its unique identification number, and the crucial property field that tracks the process that created it.

Your Phase A Submission:

[1] Your Pipeline Query:

[2] Paste Your Output Table (Identify the common `ParentProcessId` number causing the issue):

Phase B: Expose and Audit the Master Process

Now that you have discovered the malicious parent identification number from Phase A, it is time to switch your magnifying glass directly to that parent process.

Write a single pipeline command using `Get-CimInstance Win32_Process` that targets the exact ID number you discovered as the parent in Phase A. Your query must extract its key structural details by selecting exactly five specific columns: the process Name, its identification number, its executable file system path, its creation date, and the initialization text string property that reveals how the command was executed.

 **HINT:** Do not guess the property names! Pipe your query for this parent process into `Get-Member` first. Look through the available properties to find the specific field that stores the complete execution command line string used to start the process. Once you find its exact name, replace `Get-Member` with `Select-Object` to capture it alongside your other four columns. You will find an anomalous, scrambled string hidden right inside that field!

Your Phase B Submission:

[1] Your Final Pipeline Query:

[2] Paste the Complete Evidence Table Output Below:

Phase C: Bonus Activity — Decoding the Adversary

Note: This bonus activity extends slightly outside our standard lab tracking parameters, but it introduces an essential investigative mindset for analyzing real-world adversary indicators.

Look closely at the scrambled value residing inside the \$Tag variable within the initialization execution string you extracted in Phase B. Attackers frequently use this specific structural layout (notice how it ends with trailing = signs) to disguise clear text strings from automated network and security filters. Your job is to identify what encoding method was used to scramble this text and unveil the hidden message. Copy the suspicious text string you found, and use any online decoding tool (such as CyberChef or a public web decoder) to crack open the text.

What is the decoded, plain text message hidden within the malicious process property?

Answer: _____

Phase D: Plot Twist — Neutralize and Evict the Threat!

Finding the evidence is only half the job. In a real incident response scenario, the threat is still running and consuming system resources until you kill it. The final containment steps are entirely up to you! Write the commands necessary to completely clean the system. Your response must accomplish two objectives:

- **Kill the Hidden Script:** Target the hidden, malicious powershell.exe instance that is hosting the while(\$true) loop and spawning the windows.
- **Purge the Remnants:** Close down all of the remaining orphaned notepad windows littering your taskbar.

 **HINT:** You can solve this using your new Get-CimInstance filtering techniques paired with a loop structure (ForEach-Object) to cleanly target the command properties, or you can use a direct, alternative Get-Process name-matching strategy combined with Stop-Process to wipe out the remaining windows.

Your Phase D Containment Submission (Write down the commands you used to clean your system):

[1] Command(s) used to stop the simulated malware parent engine:

[2] Command(s) used to close out all residual Notepad windows:
